Локальные LLM Open API

Ветров С. В.

Содержание

1. Слайды про LLM
2. Локальные LLM
3. OpenAPI

Локальные LLM

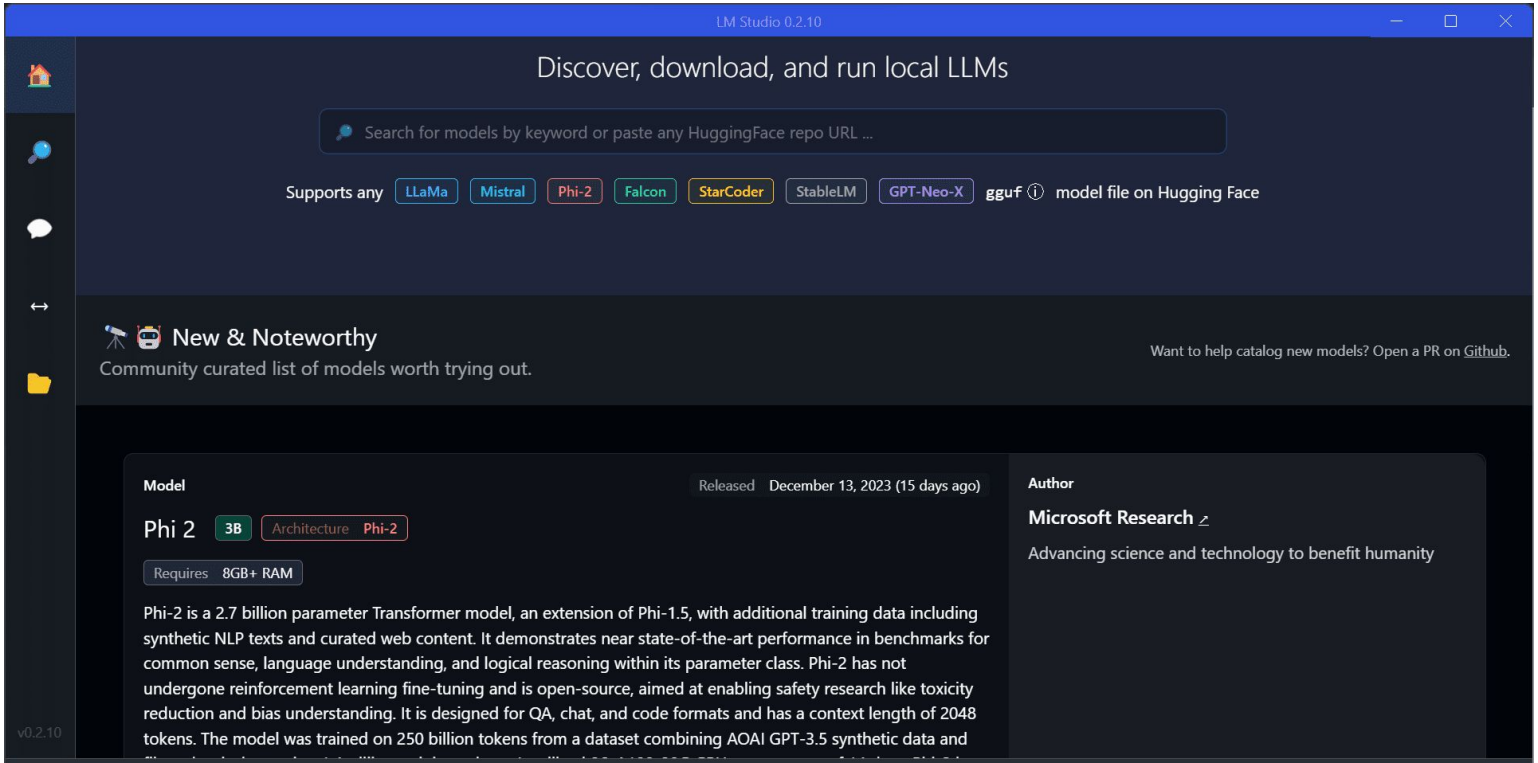
Зачем?

1. Право на свои данные
   В пользовательских соглашениях у многих сервисов заявлено право на использование и запоминание *всех* данных пользователя
   Сервис с LLM может знать о вас больше, чем поисковики или соцсети.
2. Защита данных
   Данные в локальных LLM не покидают юрисдикции
3. Полный контроль над LLM

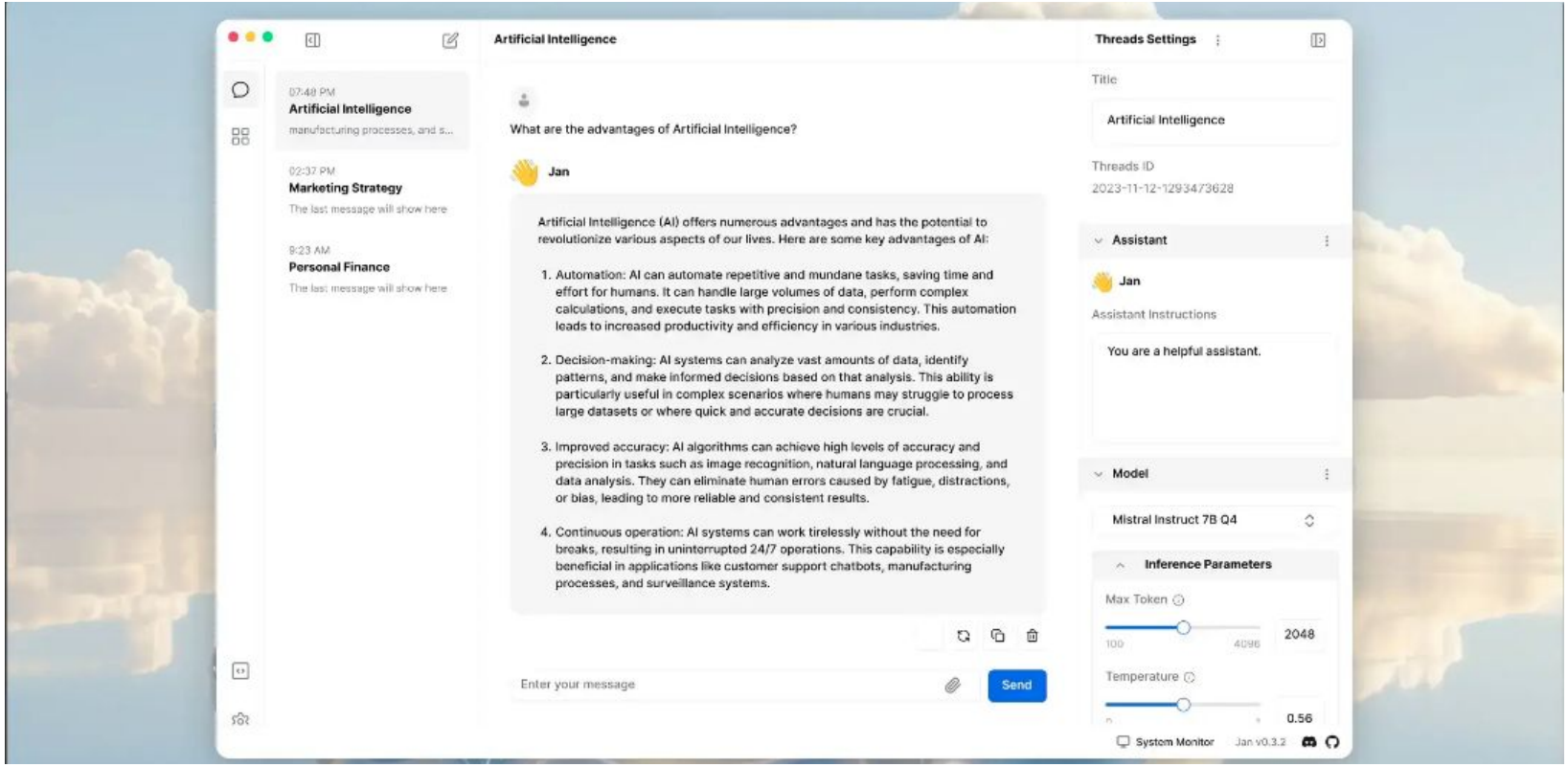
LM Studio (lmstudio.ai)

Приложение для скачивания и запуска языковых моделей.

Есть интерфейс чата и API для работы с моделями.



Jan


Artificial Intelligence
07:48 PM
Artificial Intelligence
manufacturing processes, and s...
02:37 PM
Marketing Strategy
The last message will show here
9:23 AM
Personal Finance
The last message will show here
What are the advantages of Artificial Intelligence?
Jan
Artificial Intelligence (AI) offers numerous advantages and has the potential to revolutionize various aspects of our lives. Here are some key advantages of AI:
1. Automation: AI can automate repetitive and mundane tasks, saving time and effort for humans. It can handle large volumes of data, perform complex calculations, and execute tasks with precision and consistency. This automation leads to increased productivity and efficiency in various industries.
2. Decision-making: AI systems can analyze vast amounts of data, identify patterns, and make informed decisions based on that analysis. This ability is particularly useful in complex scenarios where humans may struggle to process large datasets or where quick and accurate decisions are crucial.
3. Improved accuracy: AI algorithms can achieve high levels of accuracy and precision in tasks such as image recognition, natural language processing, and data analysis. They can eliminate human errors caused by fatigue, distractions, or bias, leading to more reliable and consistent results.
4. Continuous operation: AI systems can work tirelessly without the need for breaks, resulting in uninterrupted 24/7 operations. This capability is especially beneficial in applications like customer support chatbots, manufacturing processes, and surveillance systems.
Enter your message
Send
Threads Settings
Title
Artificial Intelligence
Threads ID
2023-11-12-1293473628
Assistant
Jan
Assistant Instructions
You are a helpful assistant.
Model
Mistral Instruct 7B Q4
Inference Parameters
Max Token
100
4096
2048
Temperature
0.56
System Monitor
Jan v0.3.2

jan.ai

ollama

ollama.com

```
# Установка в linux:
curl -fsSL https://ollama.com/install.sh | sh

# запуск (скачивание, если нужно) модели на сервере
ollama run gemma:2b
```

```
>>> what is lama?
A lama is a type of Buddhist monk or nun who follows the Tibetan Buddhist tradition.
Lamas are responsible for preserving and transmitting Buddhist teachings and practices
to other lamas and lay practitioners. They are highly respected in Tibetan Buddhism and
are known for their dedication and selflessness.

>>> what is llama?
A llama is a type of mammal belonging to the camelid family Camelidae. They are found in
the Andes Mountains of South America. Llamas are known for their large size, soft fur,
and social behavior. They are herbivores and are an important part of the Andean
ecosystem.

>>> Send a message (/? for help)
```

gemma3
The current, most capable model that runs on a single GPU.
vision 1b 4b 12b 27b
2.6M Pulls 16 Tags Updated 7 days ago

qwq
QwQ is the reasoning model of the Qwen series.
tools 32b
1M Pulls 8 Tags Updated 9 days ago

deepseek-r1
DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.
1.5b 7b 8b 14b 32b 70b 671b
29.2M Pulls 29 Tags Updated 6 weeks ago

llama3.3
New state of the art 70B model. Llama 3.3 70B offers similar performance compared to the Llama 3.1 405B model.
tools 70b
1.6M Pulls 14 Tags Updated 3 months ago

phi4
Phi-4 is a 14B parameter, state-of-the-art open model from Microsoft.
14b
1.2M Pulls 5 Tags Updated 2 months ago

llama3.2
Meta's Llama 3.2 goes small with 1B and 3B models.
tools 1b 3b
11.2M Pulls 63 Tags Updated 5 months ago

Короткая шпаргалка по ollama: github.com/ivtipm/ML/tools.md

LLM для локального запуска (открытые LLM)

huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard

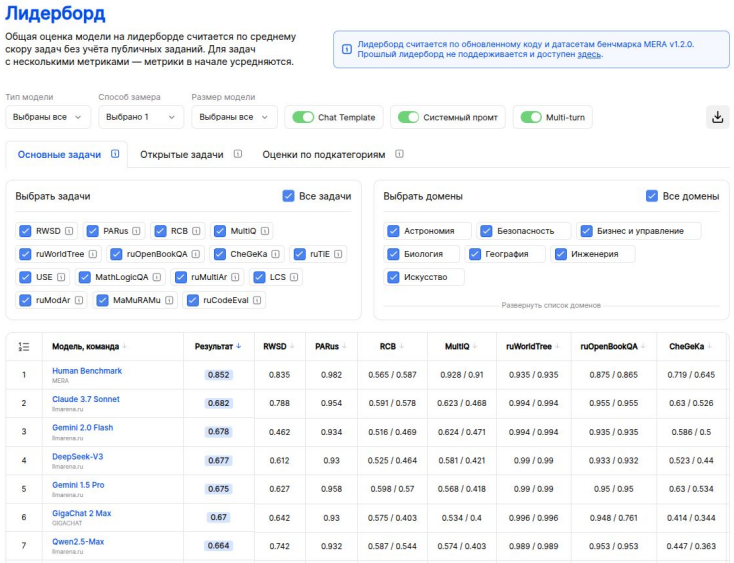
Избранные LLM доступные для локального запуска

llama3.2:1b, llama3.2:3b — одни из самых небольших моделей

Веса llama3.2 с 1.23 млн параметров в квантизации Q8 занимает 1.3 Гб

vikhr-llama-3.2-1b-instruct — дообученная на русском датасете llama-3.2-1b

Есть версии на 1 млрд. и 3 млдр. параметров.



	Модель, команда	Результат	RWSD	PARus	RCB	MultiQ	ruWorldTree	ruOpenBookQA	CheGeKa
1	Human Benchmark MERA	0.852	0.835	0.982	0.565 / 0.587	0.928 / 0.91	0.935 / 0.935	0.875 / 0.865	0.719 / 0.645
2	Claude 3.7 Sonnet llmarena.ru	0.682	0.788	0.954	0.591 / 0.578	0.623 / 0.468	0.994 / 0.994	0.955 / 0.955	0.63 / 0.526
3	Gemini 2.0 Flash llmarena.ru	0.678	0.462	0.934	0.516 / 0.469	0.624 / 0.471	0.994 / 0.994	0.935 / 0.935	0.586 / 0.5
4	DeepSeek-V3 llmarena.ru	0.677	0.612	0.93	0.525 / 0.464	0.581 / 0.421	0.99 / 0.99	0.933 / 0.932	0.523 / 0.44
5	Gemini 1.5 Pro llmarena.ru	0.675	0.627	0.958	0.598 / 0.57	0.568 / 0.418	0.99 / 0.99	0.95 / 0.95	0.63 / 0.534
6	GigaChat 2 Max GIGACHAT	0.67	0.642	0.93	0.575 / 0.403	0.534 / 0.4	0.996 / 0.996	0.948 / 0.761	0.414 / 0.344
7	Qwen2.5-Max llmarena.ru	0.664	0.742	0.932	0.587 / 0.544	0.574 / 0.403	0.989 / 0.989	0.953 / 0.953	0.447 / 0.363

mera.a-ai.ru/ru/leaderboard

Пример. Модель lakomoor/vikhr-llama-3.2-1b-instruct:1b

```
>>> /show info
  Model
      parameters        1.2B
      quantization      Q4_0
      arch              llama
      context length    131’072
      embedding length  2048

  Parameters
      num_ctx           4096
      stop              "◁start_header_id▷"
      stop              "◁end_header_id▷"
      stop              "◁eot_id▷"
      stop              "◁reserved_special_token"
      temperature       0.2

  System
      Вы - Vikhr, помощник с искусственным интеллектом, созданный компанией Vikhr models, чтобы быть
полезным, безобидным и честным.
```

Количество параметров, b — billions (миллиарды)

Максимальный размер контекста в токенах

Размерность вектора (эмбеддинга) токена

Фактический используемый размер контекста (в токенах). Можно увеличить в настройках

```
/set parameter num_ctx <value>
```

ollama.com/lakomoor/vikhr-llama-3.2-1b-instruct/tags

Пример

Название модели

Количество параметров b — billion

Доп. информации о размере или квантизации (как здесь). fp16 - веса хранятся как 16-битные веществ числа

lakomoor/vikhr-llama-3.2-1b-instruct:16fp

На ollama.com так указывается аккаунт, с которого выложена модель

Название модели, на основе которой обучение (fine tune) текущая модель

Модель обучена выполнять инструкции (не просто продолжать текст)

ollama.com/lakomoor/vikhr-llama-3.2-1b-instruct/tags

Квантизация (Quantization)

Квантизация LLM — это процесс уменьшения размера модели путём сокращения точности числовых представлений параметров (весов) модели.
Это повышает скорость работы и объём требуемой памяти.

Виды:

- FP32 — я исходная точность
- FP16
- INT8, обычно требует дообучения модели

- Dynamic Quantization — квантизация только во время выполнения, но веса хранятся в FP32
- Постобучающая квантизация
- Квантизация с дообучением (Quantization-Aware Training, QAT)

Квантизация (Quantization)

- K-Quants
  Например Q3_K_M — квантизация до 3 бит (примерно); M — Medium
- IQ-Quants

Использование LLM напрямую в Python

```
from transformers import AutoModelForCausalLM, AutoTokenizer


# Загрузка модели и токенизатора
model_name = "Vikhrmodels/Vikhr-Llama-3.2-1B-instruct"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

input_text = "Напиши очень краткую рецензию о книге гарри поттер."

# Токенизация и генерация текста
input_ids = tokenizer.encode(input_text, return_tensors="pt")
output = model.generate(
  input_ids,
  max_length=1512,   temperature=0.3,   num_return_sequences=1,
  no_repeat_ngram_size=2,   top_k=50,   top_p=0.95,   )

# Декодирование и вывод результата
generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
print(generated_text)
```

Запуск LLM, обращение через requests

```
ollama –help

ollama serve
http://localhost:11434

ollama pull <model_name>

ollama list

ollama run


>>> /?

>>> clear
```

Список моделей

http://localhost:11434/api/tags

или

curl -X GET http://localhost:11434/api/tags

Обращение:

```
curl -X POST
http://localhost:11434/api/generate \
-H "Content-Type: application/json" \
-d '{
  "model": "llama3.2:1b",
  "prompt": "tell me a joke about three
programmers and one doctor",
  "stream": false
}'
```

Выдавать ответ частями (true) или сразу (false)?

Python: requests

```python
import requests
# import json

BASE_URL = "http://localhost:11434/api"

# 1. Получение списка моделей
response = requests.get(f"{BASE_URL}/tags")
if response.status_code == 200:
    print( response.json() )        # → dict
else:
    print("Ошибка при получении моделей:", response.status_code)
```

Python: requests

```python
MODEL = 'llama3.2:1b'
payload = {
    "model": MODEL,
    "prompt": "How many chromosomes does human have?",
    "stream": False
}

response = requests.post(f"{BASE_URL}/generate", json=payload)
if response.status_code == 200:
    print( response.json()['response'] )
else:
    print("Ошибка при генерации текста:", response.status_code)
```

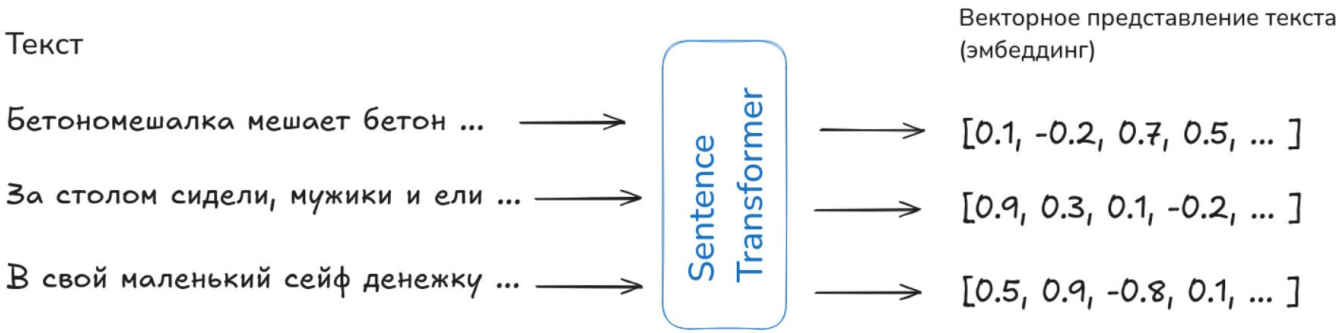
Python: ollama

```python
import ollama

mes = 'Please translate from latin: Non dies sine committere'

response = ollama.chat(model='llama3.2:1b',
                       messages=[{'role': 'user', 'content': mes}])

print(response['message']['content'])
```

Python: langchain

```python
from langchain_community.chat_models import ChatOllama
from langchain.schema import HumanMessage

# адрес и порт сервера Ollama
ollama_base_url = "http://localhost:11434"

# Инициализация модели
llm = ChatOllama(model="llama3.2:1b", base_url=ollama_base_url)

# Функция для получения ответа от модели
def get_response(user_input):
    messages = [HumanMessage(content=user_input)]
    return llm.invoke(messages).content

# Пример использования
user_input = "What is LD50 of coffee?"
response = get_response(user_input)
print("Ответ:", response)
```

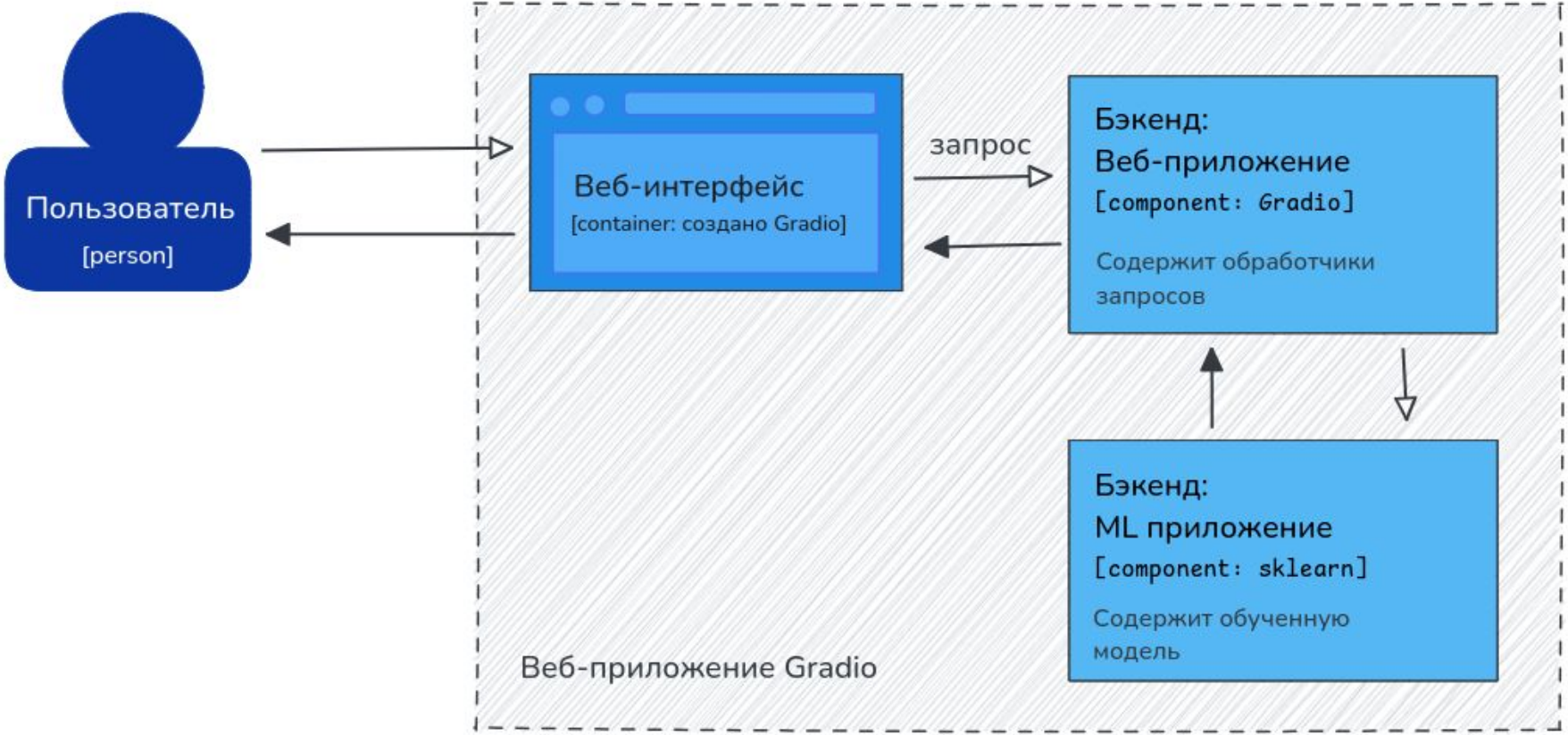
Sentence Transformers


Текст
Бетономешалка мешает бетон ...
За столом сидели, мужики и ели ...
В свой маленький сейф денежку ...
Sentence Transformer
Векторное представление текста (эмбеддинг)
[0.1, -0.2, 0.7, 0.5, ...]
[0.9, 0.3, 0.1, -0.2, ...]
[0.5, 0.9, -0.8, 0.1, ...]

colab.research.google.com/drive/1XDLKd42U5UcDSfwXtDgR3uJuFND9xCvS

API

Минимальное приложение на Gradio



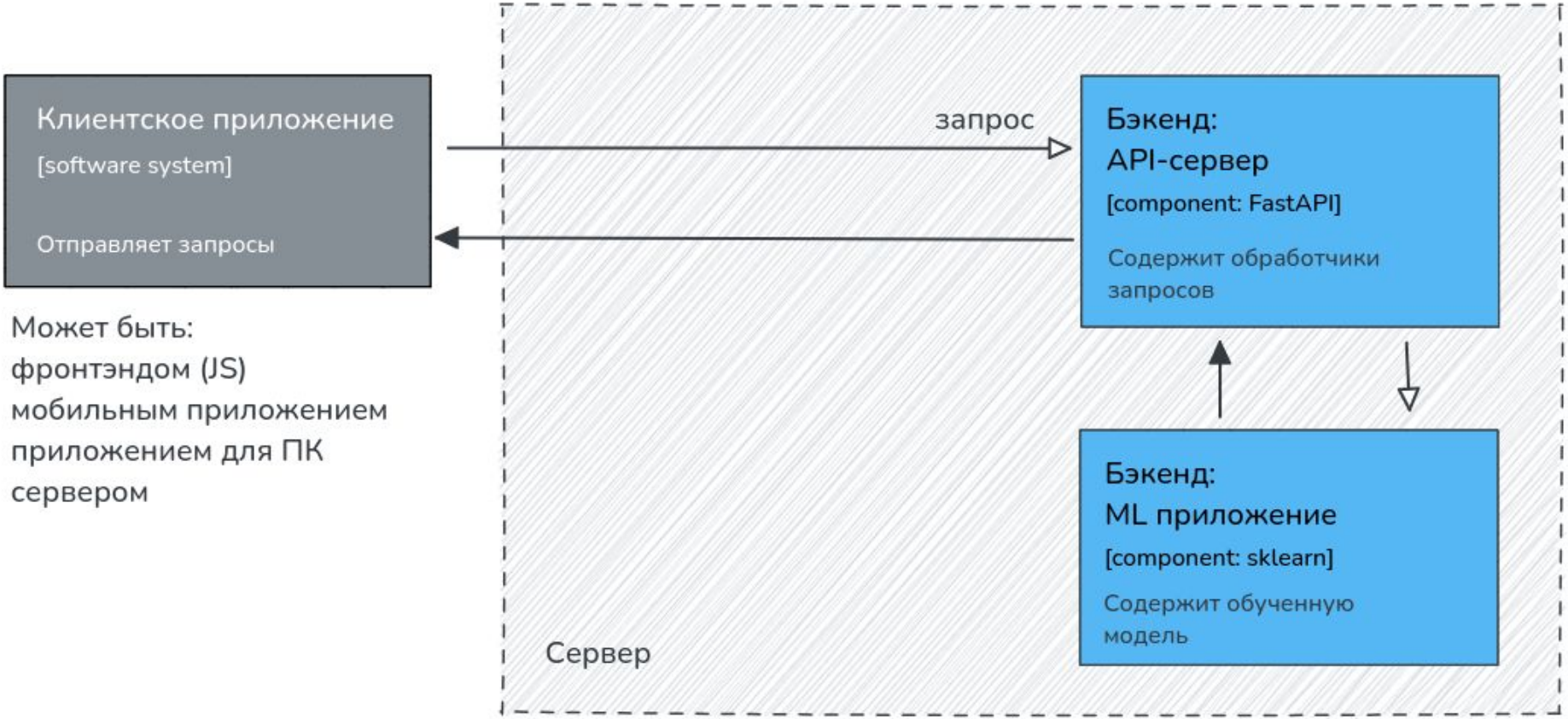
API

API (Application Programming Interface) — это общий интерфейс для взаимодействия любых программных (или аппаратных) компонентов.

API задаёт правила и договорённости о том, как один модуль (класс, сервис, библиотека, программа или устройство) может использовать функции или данные другого:

- Методы/функции: какие операции доступны.
- Формат данных: в каком виде передаются входные и выходные параметры (числа, строки, структуры, JSON и др.).
- Протоколы и соглашения: как устанавливается соединение (например, вызов функции в коде, вызов по HTTP/HTTPS, обмен сообщениями по MQTT и др).

Минимальное приложение на FastAPI



Может быть:
фронтэндом (JS)
мобильным приложением
приложением для ПК
сервером

Зачем использовать API?

Отделение модели и остальных компонентов системы.
Модель и её окружение находятся в одном сервисе, а фронтенд/мобильные клиенты или другие сервисы просто делают запросы. Это даёт независимое развертывание и масштабирование.

Унификация интерфейса. Клиентам не нужно знать внутренней структуры модели — достаточно стандартизованного request/response контракта.

Разные клиенты (веб/мобильные/скрипты) получают единый интерфейс.

Контроль и наблюдаемость. Через API проще логировать входы/выходы, метрики задержки (latency), и отслеживать деградацию качества модели (drift).

Безопасность и управление доступом. Можно централизованно ограничивать доступ, ставить аутентификацию/авторизацию, лимиты запросов (rate limiting).

Версионирование и откат. Через API проще публиковать несколько версий модели и быстро откатываться на стабильную версию при проблемах.

Примеры открытых (public) API

Wttr.in (github.com/chubin/wttr.in, wttr.in/:help) — простой API, не требует регистрации
GET https://wttr.in/Chita_Ru?format=3

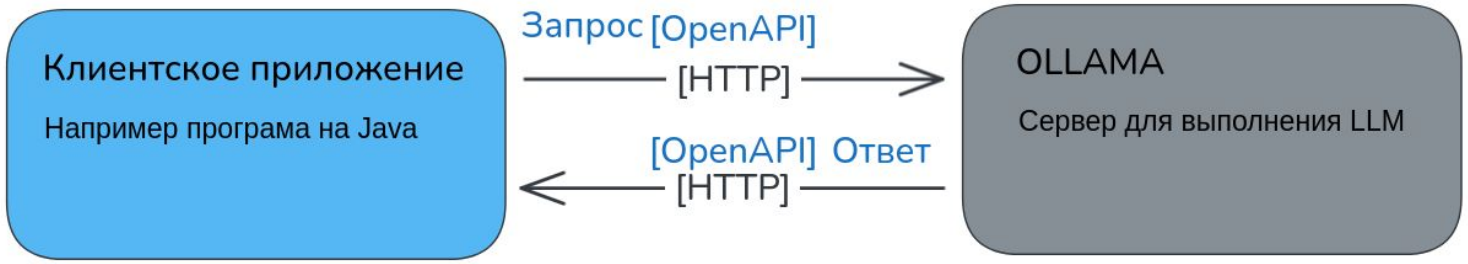
OpenWeatherMap (погода)
GET
https://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_API_KEY

Hugging Face Inference API (ML-инференс)
POST
https://api-inference.huggingface.co/models/{MODEL}
с Authorization: Bearer <TOKEN>

Список открытых API: https://github.com/public-apis/public-apis

OpenAPI

OpenAPI — машинно-читаемый контракт API (JSON/YAML), который описывает endpoints, схемы запросов/ответов, security; позволяет генерировать docs/clients/tests.



HTTP — протокол передачи данных

OpenAPI — стандарт для API, реализуется поверх HTTP

HTTP

Модель				
Уровень (layer)		Тип данных (PDU)		Примеры
Host layers	7. Прикладной (application)	Данные	Доступ к сетевым службам	HTTP, FTP, POP3, SMTP, WebSock
	6. Представления (presentation)		Представление и шифрование данных	ASCII, EBCDIC, SSL, gzip
	5. Сеансовый (session)		Управление сеансом связи	RPC, PAP, L2TP, gRPC
	4. Транспортный (transport)	Сегменты (segment) / Датаграммы (datagram)	Прямая связь между конечными пунктами и надёжность	TCP, UDP, SCTP, Порты
Media layers	3. Сетевой (network)	Пакеты (packet)	Определение маршрута и логическая адресация	IPv4, IPv6, IPsec, AppleTalk, ICMP
	2. Канальный (data link)	Биты (bit)/ Кадры (frame)	Физическая адресация	PPP, IEEE 802.22, Ethernet, DSL, ARP, сетевая карта.
	1. Физический (physical)	Биты (bit)	Работа со средой передачи, сигналами и двоичными данными	USB, RJ («витая пара», коаксиальный, оптоволоконный), радиоканал

Протокол HTTP используется для передачи данных веба (между браузером и веб-сервером) и для передачи данных между программами (т.е. для реализации API)

Структура HTTP Запроса (request)

*Содержит адрес — **эндпоинт**, к которому отправляется запрос*

На стороне сервера обычно для каждого эндпоинта подготовлен специальный обработчик

*Помимо адреса указывается **метод** — обозначение действия, которое клиент хочет совершить с данными*

GET — получение данных; Параметра запроса данных передаются прямо в адресе. Например /weather?city=Chita

POST — Отправка данных (не исключает получение данных в ответ).
*Данные передаются в **теле запроса (HTTP body)***

REQUEST

```
GET http://127.0.0.1:5500/styles/navigation.css HTTP/1.1
```

HTTP request line

```
HOST: 127.0.0.1:5500
Accept: text/css,*/*;q=0.1
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
User-Agent: Mozilla/5.0 (Windows NT 10.0;
Win64; x64; rv:102.0) Gecko/20100101 Firefox/102.0
Connection: keep-alive
<CRLF>
```

HTTP headers

HTTP body (empty)

Заголовки (headers) содержат техническую информацию о запросе

HTTP Request Methods

GET	POST	PUT	DELETE	PATCH	HEAD
retrieve data from server	add data to an existining file or resource	update(replace) an existing file or resource in server	delete data from server	update a resource partially (modify)	retrieve the resource's headers

- **CONNECT** is used to open a two-way socket connection to the remote server;
- **OPTIONS** is used to describe the communication options for specified resource;
- **TRACE** is designed for diagnostic purposes during the development.
- **HEAD** retrieves the resource's headers, without the resource itself.

HTTP

- Запросы (HTTP Requests). Основой запроса является HTTP-заголовок.

- Ответы (HTTP Responses) — сообщения, которые сервер отправляет в ответ на клиентский запрос.

JSON

Часто для представления данных, передающихся по HTTP между программами используется формат JSON

JSON

Запись

ключ

значение – строковый литерал

```
{
    "firstName": "Иван",
    "lastName": "Иванов",
    "address": {
        "streetAddress": "Московское ш., 101, кв.101",
        "city": "Ленинград",
        "postalCode": 101101
    },
    "phoneNumbers": [
        "812 123-1234",
        "916 123-4567"
    ]
}
```

числовой литерал

массив

OpenAPI

OpenAPI — это формализованная спецификация и экосистема инструментов, которая предоставляет интерфейс описания HTTP API.

- OpenAPI не зависит от языков программирования
- Позволяет генерировать исходный код для библиотек клиентских приложений
- Позволяет генерировать текстовую документацию для пользователей
- Позволяет генерировать варианты тестирования и другие элементы на основе спроектированной спецификации для некоторого сервиса

Можно генерировать спецификацию и документацию из кода.

Примеры

Запросы на Java —
github.com/ivtipm/BigDataLanguages/blob/main/Java/requests.md

Curl, браузер, hoppscotch, postman

Curl — простая консольная утилита для HTTP запросов

Пример POST запроса:

```
curl http://localhost:11434/api/generate
     -d '{"model": "llama3.2:3b", "prompt": "What is LLM?"}
```

Простые GET-запросы легко протестировать в браузере

hoppscotch.io — программа и онлайн-сервис для тестирования API

Пример. Python. GET запрос

```
response = requests.get("https://wttr.in/Chita_ru?format=2")
if response.status_code == 200:
    print("Успех!")
    print(response.json())
else:
    print("Ошибка:", response.status_code)
```

Пример. Python. POST запрос

```
response = requests.post("https://example.com",
                         json={"key": "value"})
```

FastAPI

FastAPI — один из популярных пакетов для реализации сервера по спецификации OpenAPI

Пример: github.com/ivtipm/ML/tree/main/examples/FastAPI

Альтернативы FastAPI

Flask — минималистичный WSGI веб-фреимворк

Django REST Framework (DRF) — мощный инструмент для построения REST API поверх Django

Starlette — более легковесный чем FastAPI, лежит в основе FastAPI

Sanic

aiohttp

Falcon

Tornado

Connexion

Bottle, CherryPy и другие микрофреймворки

GraphQL-сервисы (например, Ariadne, Strawberry)